

With the command name and help argument, *gdb* displays a short paragraph on how to use that command.

A buffer overflow

A buffer overflow happens when a program tries to store more data than the actual size of the buffer. In such a case, the data overflows from the buffer, which leads to overwriting of the adjacent memory fragments of the process, as well as overwriting of the values of the IP (Instruction Pointer) or BP (Base Pointer) or other registers. This causes exceptions and segmentation faults, leading to other errors.

The problem given in the code snippet below will give you an idea about buffer overflows.

```
#include<stdio.h>
int main()
{
    char buffer[50];
    buffer[60]='a';
    return 0;
}
```

When you compile the above problem, the compiler will not throw you an error because there is no automatic bound checking on the buffer. But when you try to see the output of the program, it throws up a segmentation fault.

In buffer overflow attacks, the hacker tries to take advantage of extra memory segments for other operation instruction sets to inject malicious arbitrary code such as shell codes, and the pre-determined program behaviour is changed eventually. To exploit buffer overflows, you need to have some idea of assembly code instructions and you should get control over the *eip* register. Getting control over *eip* is very simple—you just need to know how the stack is framed and know where the *eip* register is located. Getting control of *gdb* also helps you to find the *eip* register. Once you get control over *eip* you can return to any point in the code and get arbitrary things like the shell.

A buffer overflow also occurs due to some vulnerabilities in the problem. Normally, buffer overflow vulnerabilities are found through source code analysis, or by reverse engineering application binaries.

With the help of this small problem, let us look at how a buffer overflow could possibly occur.

```
#include<stdio.h>
#include<string.h>
void function(char *string)
{
    char buffer[50];
    strcpy(buffer, string);
}
int main(int argc, char **argv)
{
```

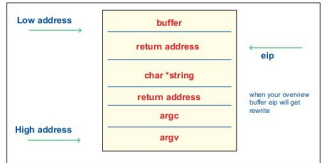


Figure 3: Buffer overflow

```
function(argv[1]);
return 0;
}
```

Figure 3 clearly describes the above problem.

In the above problem, if the input is less than 50 characters, then the program will execute normally. When 50 characters are exceeded, the compiler throws up a segmentation fault. The above problem describes how the vulnerability *strcpy* leads to an overflow.

To get started up with buffer overflows, it would be good if you start up with *pic0ctf*. Once you are familiar with it, you can ‘smash the stack’.

Shell code

In most of the binary exploitation problems, we just have to capture the shell, so we need to know a little bit about how to write shell code. As of now, we can modify the return address (which is just the address of *eip*) by just overflowing the buffer. In most cases, you just need to spawn the shell. From the shell, you can execute the command as you like. With the help of this small code snippet in C, you will get the shell.

```
#include<stdio.h>
int main()
{
    system("/bin/sh");
}
```

When you compile the above code in a terminal, you will get a shell. Writing the code in C language is simple, but when you need to inject it in a buffer, it should be in ASCII as you cannot inject the code in C into the buffer. It is not that necessary to learn to write shell code in ASCII because of online availability. The online resource for shell code is <http://shell-storm.org/shellcode/> 

By: Rakesh Paruchuri

The author is a security enthusiast.